Atividade de Pesquisa – Três linguagens de programação

Nome : Rhian Dias

Professor: Gioliano Bertoni

Curso: Desenvolvimento de Sistemas

Parte 1

Linguagem 1

Nome da linguagem: Python

Características: Linguagem de alto nível, interpretada e de sintaxe simples

Aplicabilidade: Desenvolvimento Web, Ciência de Dados, Machine Learning/Desenvolvimento de IA, Automação, Criação de scripts e testes de software

Pontos fortes/ vantagens: Fácil de aprender, fácil de usar, grande quantidade de bibliotecas padrão e frameworks, grande comunidade de suporte, cross-plataforma

Pontos fracos: Dependendo do projeto pode consumir muita memória, sua tipagem dinâmica facilita o surgimento de erros em tempo de execução

Mercado de Trabalho: Amplamente procurado em cargos de TI, alta demanda

Linguagem 2

Nome da linguagem: C++

Características: Linguagem compilada, de alto desempenho e com controle detalhado de recursos

Aplicabilidade: Desenvolvimento de jogos, desenvolvimento de sistemas operacionais, sistemas embarcados, aplicações gráficas, robótica e simulações científicas

Pontos fortes/ vantagens: alta performance, controle detalhado da memória e do hardware, orientada a objeto, largamente usada em bibliotecas C/C++

Pontos fracos: curva de aprendizado acentuada, gerenciamento de memória e ponteiros exigem conhecimento e cuidado, erros como acessos inválidos à memória podem ser difíceis de diagnosticar

Mercado de Trabalho: Procurada geralmente em nichos técnicos, esperada como linguagem base para maioria dos cargos de TI nessa habilitação (técnico); mercado mais consolidado

Linguagem 3

Nome da linguagem: Rust

Características: linguagem compilada, focada em desempenho, segurança de memória e concorrência

Aplicabilidade: infraestrutura de software, ferramentas de sistemas, serviços de baixa latência, segurança, aplicações de rede, desenvolvimento de sistemas, webassembly, aplicativos de performance crítica

Pontos fortes/ vantagens: segurança da memória (previne erros de memória em tempo de compilação), custo zero em abstrações, facilidade de criar programas concorrentes mais seguros, alta performance

Pontos fracos: curva de aprendizado considerável (necessidade de lidar com conceitos como ownership e borrowing), compilador rigoroso (exige mais esforço inicial)

Mercado de Trabalho: Não aparece em muitos cargos de TI e é procurada de forma especializada; especialização em alta

Parte 2 – Quadro Comparativo

CRITÉRIO	Python	C++	Rust
Características principais	Alto nível, interpretada e sintaxe simples	Compilada, alto desempenho e controle de memória	Compilada, alto desempenho e segurança de memória
Vantagens	Fácil aprendizado, produtividade e muitas bibliotecas padrão	Eficiência, controle de recursos e ecossistema consolidado	Segurança de memória, desempenho e concorrência segura
Aplicações mais comuns	IA, Ciência de Dados, Automação e Web	Jogos, robótica, sistemas embarcados e simulações	Infraestrutura, redes, segurança e sistemas embarcados
Pontos fortes	Versatilidade, rapidez no desenvolvimento e ampla comunidade	Velocidade, controle do hardware e maturidade	Desempenho, confiabilidade e prevenção de erros de memória
Pontos fracos/ limitações	Menor desempenho em tarefas intensivas, maior consumo de memória em alguns cenários	Aprendizado complexo, maior risco de erros de memória	Curva de aprendizado acentuada, menor quantidade de vagas
Áreas ou cargos no mercado	Desenvolvedor Back-End, Analista de Dados, Engenheiro de Dados, Especialista em IA	Engenheiro de Software, Desenvolvedor de Jogos, Engenheiro de Sistemas Embarcados	Engenheiro de Sistemas, Desenvolvedor de Infraestrutura, Especialista em Software de Baixa Latência
Conhecimentos complementares	SQL, Git, APIs, bibliotecas de dados e frameworks web	Algoritmos, estrutura de dados, sistemas operacionais, memória e depuração	Ownership e borrowing, concorrência, sistemas operacionais e redes
Exemplo de projeto adequado	Chatbot com IA, sistemas de análise de dados, API web	Engine de jogos, simulador físico, sistemas de controle robótico	Servidor de alta concorrência, ferramenta de segurança ou serviço de rede eficiente

Exemplos usando cada linguagem usada na pesquisa:

- Algoritmo de Busca Binária de modo recursivo em Python:

```python
def busca_binaria_recursiva(lista, inicio, fim, alvo):
    # Caso base: se o intervalo for inválido, o elemento não está na lista
    if inicio > fim:
        return -1

    # Encontra o índice do meio
    meio = inicio + (fim - inicio) // 2

    # Se o elemento do meio for o alvo, retorna o índice
    if lista[meio] == alvo:
        return meio

    # Se o alvo for menor que o meio, busca na metade esquerda
    elif lista[meio] > alvo:
        return busca_binaria_recursiva(lista, inicio, meio - 1, alvo)

    # Se o alvo for maior que o meio, busca na metade direita
    else:
        return busca_binaria_recursiva(lista, meio + 1, fim, alvo)


# Exemplo de uso:
numeros = [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]
alvo = 23

resultado = busca_binaria_recursiva(numeros, 0, len(numeros) - 1, alvo)
print(f"Elemento encontrado no índice: {resultado}")  # Saída: 5
```

- Algoritmo de Busca Binária de modo recursivo em C++:

```cpp
#include <iostream>
#include <vector>

// Função auxiliar recursiva
int buscaBinariaRecursiva(const std::vector<int>& arr, int esquerda, int direita, int alvo) {
    if (esquerda > direita) {
        return -1; // Elemento não encontrado
    }

    int meio = esquerda + (direita - esquerda) / 2;

    if (arr[meio] == alvo) {
        return meio; // Elemento encontrado no índice 'meio'
    }

    if (arr[meio] > alvo) {
        return buscaBinariaRecursiva(arr, esquerda, meio - 1, alvo); // Busca na metade esquerda
    }

    return buscaBinariaRecursiva(arr, meio + 1, direita, alvo); // Busca na metade direita
}

// Função principal de interface
int buscaBinaria(const std::vector<int>& arr, int alvo) {
    return buscaBinariaRecursiva(arr, 0, arr.size() - 1, alvo);
}

int main() {
    std::vector<int> dados = {2, 5, 8, 12, 16, 23, 38, 56, 72, 91};
    int alvo = 23;

    int resultado = buscaBinaria(dados, alvo);

    if (resultado != -1) {
        std::cout << "Elemento " << alvo << " encontrado no índice: " << resultado << std::endl;
    } else {
        std::cout << "Elemento " << alvo << " não encontrado no vetor." << std::endl;
    }

    return 0;
}
```

- Algoritmo de Busca Binária de modo recursivo em Rust:



```rust
fn busca_binaria_recursiva<T: Ord>(slice: &[T], alvo: T) -> Option<usize> {
    fn aux<T: Ord>(slice: &[T], alvo: &T, inicio: usize) -> Option<usize> {
        if slice.is_empty() {
            return None;
        }

        let meio = slice.len() / 2;
        match slice[meio].cmp(alvo) {
            std::cmp::Ordering::Equal => Some(inicio + meio),
            std::cmp::Ordering::Greater => aux(&slice[..meio], alvo, inicio),
            std::cmp::Ordering::Less => aux(&slice[meio + 1..], alvo, inicio + meio + 1),
        }
    }

    aux(slice, &alvo, 0)
}

fn main() {
    let numeros = [1, 3, 5, 7, 9, 11, 13, 15];
    let alvo = 7;

    match busca_binaria_recursiva(&numeros, alvo) {
        Some(indice) => println!("Elemento {} encontrado no índice {}", alvo, indice),
        Sign_or_none => println!("Elemento não encontrado"),
    }
}
```